crosstide

The Agentic Ways of Working Playbook

A new way to build product and delivery value even faster

Product Operating Rhythm Today

Aligning/Planning

Strategic Layer Planning

3 - 5 Year Strategic Horizon reviewed bi-annually

Annual Strategic Roadmap Review

(Owned at Exec Level sets direction for the organisation)

Monthly Board Level Mission Control

Strategic Outcomes

Coordination Layer - Tribe Level

Pipeline Review

Tribe Quarterly Planning

Crosstribe Alignment

12 Month Outcomes
reviewed quarterly to align with Strategy

(Owned by Business Lead/Head of Product/SEMs)

Monthly Product Showcase

Fortnightly Prod / Engineering

Tribe Outcomes / Opportunities / Initiatives

Operational Layer - Squad Level

Squad Quarterly Planning

Sprint Planning

Daily Stand Up

6 month roadmap owned by PMs reviewed quarterly

(Delivery Plan owned by squads reviewed fortnightly)

Sprint Review

Sprint Retrospective

Epics / Stories

Learning / Reporting

Delivery Models: Human vs AI Assisted vs Agentic

In the the Agentic Software Development Lifecycle:

- Agents work, not just assist
- Actions are traceable, auditable, and controlled
- Humans own decisions, risk, and outcomes

The table outlines the different Delivery Models compare: Human vs AI Assisted vs Agentic

Model	Role of AI	Human Role	Characteristics	Risks
Human Development	None	Do all the work	Manual, slower, high control, ceremony orientated	Scaling has limitations and labour intensive
AI-Assisted	Supports tasks (e.g. coding, writing)	Drives all decisions	Productivity uplift	Inconsistent usage, low traceability
Agentic	Executes defined roles autonomously	Owns outcomes & governance	Scalable, structured, artefact-driven	Requires strong controls, and agreement on level of agent autonomy

Non-Agentic Operating Model vs Agentic Operating Model

Non-Agentic Model	Agentic Model
Work is coordinated via meetings	Artefacts coordinate work
Planning gates execution	Planning filters execution
Sprints batch delivery	Continuous flow
Humans manage flow	Orchestration manages flow
Static cadence	Multi-speed system

Possible Agentic Powered Enterprise Operating Rhythm

Alignment through artefacts and prioritisation

Strategic Layer - Strategic Outcomes

3 Year Strategic Horizon
Reviewed continuously and as required based on signals and rapid learning loops

Owned at Exec Level
Continuously steers outcomes based on real-time signals
Defines risk appetite and guardrails for agent autonomy

Coordination Layer - Tribe Outcomes (Opportunities and Intiatives)

12 Month Outcomes
Dynamic Portfolio Management in line with Strategy / Goals

Owned by cross functional tribe leadership
Dynamically manages and reprioritises opportunities / initiatives
Provides real-time portfolio visibility, risk management

Operational Layer - Squad Level

Continuous planning

AI agents execute work continuously
Humans validate, approve, and remain accountable for outcomes
Quality, compliance, and control enforced through embedded validation and approval gates

Continuous telemetry, reporting, and learning

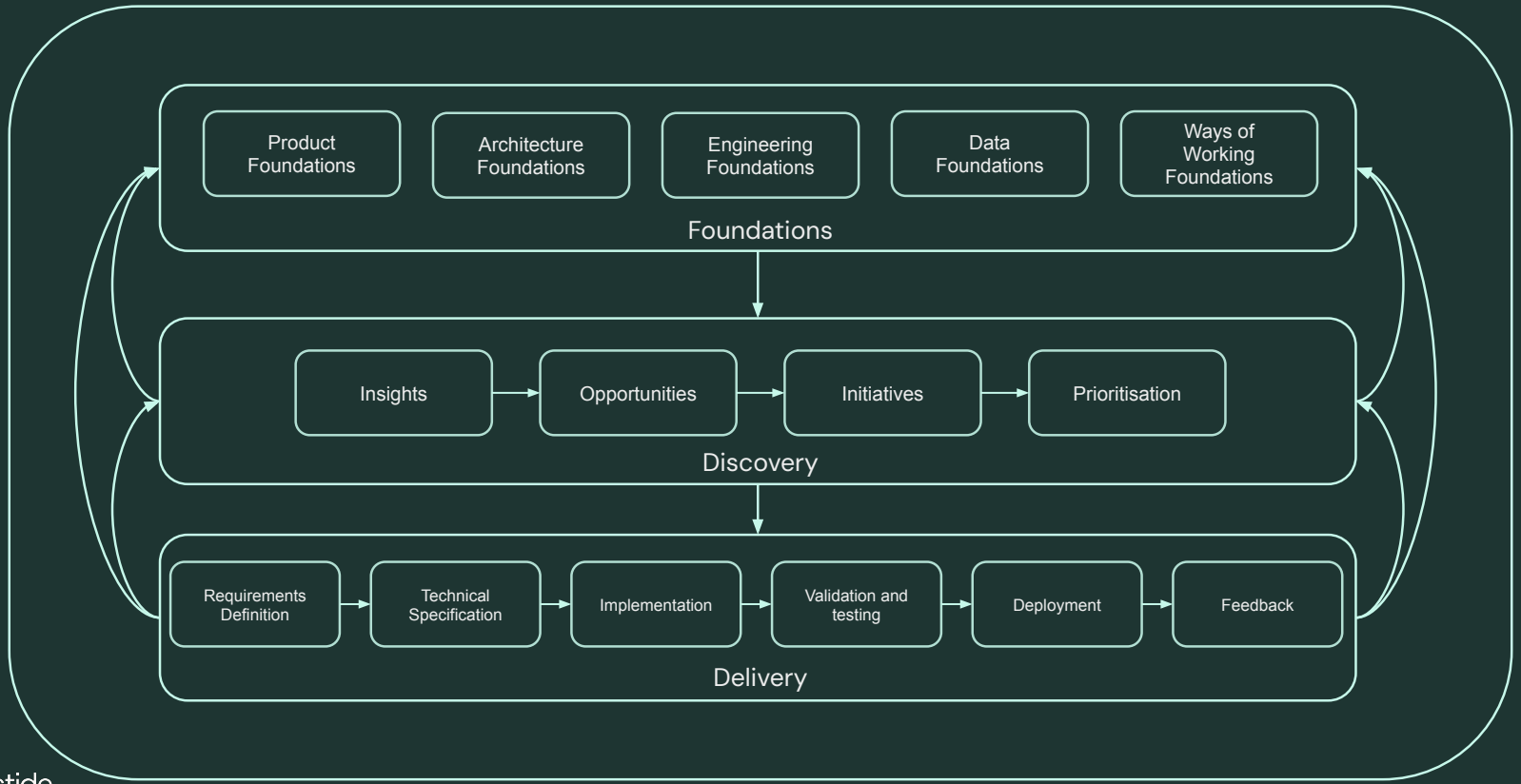
Decisions, artefacts, and changes are traceable from strategy through to delivery and outcomes

Agility enabled

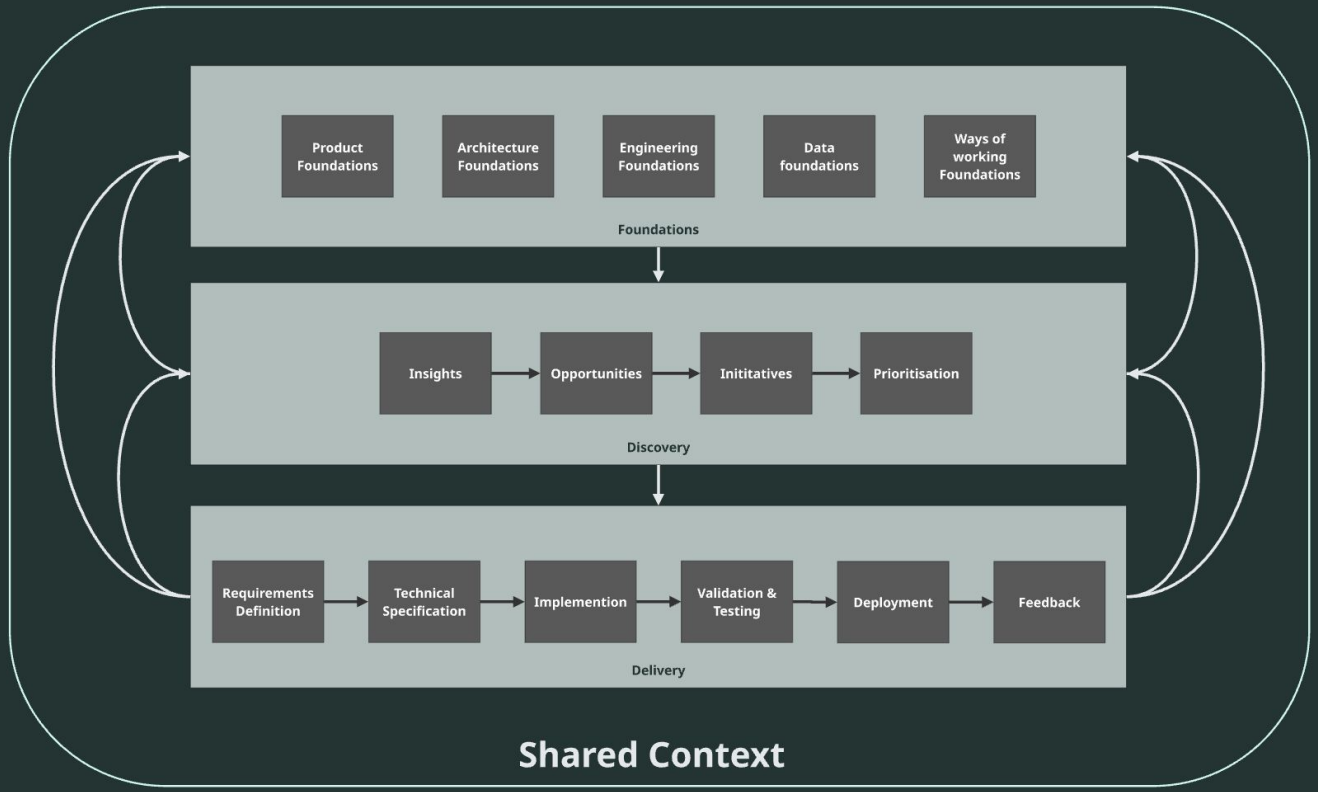
An agentic powered organisation has the potential to enable enhanced agility by removing the constraints informed by slow decision making and governance and coordination over head

Decision Area	Traditional Cadence	Continuous Future State
Strategic Priority Shifts	Annual / bi-annual	Prompted by market, competitor, or regulatory signal changes
Investment Allocation	Annual budgeting cycle	Funding moved incrementally toward highest-value outcomes
Risk Appetite / Controls	Periodic governance reviews	Adjusted when operational or regulatory risk indicators move
Portfolio Performance Intervention	Quarterly reviews	Immediate intervention when major bets underperform - reduce waste by stopping sooner.
Outcome Target Adjustment	Annual planning	Revised when assumptions change

Flow of Value and the Agentic Powered SDLC



Flow of Value and the Agentic Powered SDLC



Principles that underpin the Agentic Ways of Working

1. Outcomes over outputs
2. Cross-functional leadership at the different flight levels
3. Continuous planning at different flight levels
4. Continuous learning and discovery
5. Small, empowered, cross-functional, autonomous team
6. Be agile, stay lean
7. Leverage data
8. Engineering excellence is an enabler for faster value delivery
9. Shared context and transparency
10. Human accountability at scale
11. Safe, adaptable systems

1. Outcomes over outputs
Success is measured by value delivered for users and the business, with technology helping accelerate progress toward meaningful outcomes.

2. Cross-functional leadership at the different flight levels
Product, technology, and delivery perspectives combine at every level to improve decisions, trade-offs, and aligned execution.

3. Continuous planning at different flight levels
Planning is continuous across strategic, coordination, and operational levels, adapting quickly as evidence, priorities, and opportunities change.

4. Continuous learning and discovery
Teams continuously learn about customers, problems, and solutions to reduce uncertainty, focus on high-value work, and improve outcomes faster.

5. Small, empowered, cross-functional, autonomous teams
Stable teams with clear ownership and the right mix of capabilities are empowered to solve problems, using technology to extend capacity, speed, and effectiveness.

6. Be agile, stay lean
Small feedback loops enable rapid learning. Work is delivered iteratively in small batches, reducing waste and enabling predictable flow and adaptation.

7. Leverage data
Decisions are guided by timely data and evidence to challenge assumptions, improve performance, and focus effort where it matters most.

8. Engineering excellence is an enabler for faster value delivery
Strong engineering practices, automation, and quality built in enable faster, safer, and more scalable delivery.

9. Shared context and transparency
A clear, shared view of priorities, constraints, and progress enables alignment, traceability, and effective collaboration across teams and systems.

10. Human accountability at scale
As delivery scales through technology, accountability for decisions, outcomes, and risks remains clear and owned by people.

11. Safe, adaptable systems
Systems are designed for safe, incremental change so teams can respond quickly, improve continuously, and evolve with confidence.

Agentic Delivery Guardrails

1. Human Accountability and Control
People remain accountable for all decisions, outcomes, and risks, regardless of how much work is performed by AI agents.

2. Artefact-Driven Delivery and Shared Context for Humans and AI Agents
All work is defined, executed, and governed through structured, version-controlled artefacts that act as the shared source of truth for both humans and AI agents.

3. Governed and Bounded AI Agent Autonomy
AI agents operate within clearly defined roles, permissions, and limits, with explicit escalation to humans, and autonomy is only increased when reliability and control are proven.

4. Continuous Validation, Data-Informed Learning, and Adaptation
AI-generated outputs are continuously validated against source artefacts, with decisions guided by data and feedback loops used to improve outcomes and agent behaviour.

5. Observable, Secure, and Controlled AI-Assisted Operations
All AI agent actions, decisions, and outputs are fully traceable, auditable, and secured, enabling safe deployment, monitoring, and rollback.

6. Continuous Improvement and Evolution of the Agentic System
AI agents, workflows, and models are continuously reviewed and evolved in a controlled and governed way to keep pace with changing technology and business needs.

JACK - PRODUCT Foundations AI SLIDE

Notes:

- Need a slide to introduce the Discovery/Continuous Discovery Phase (So could use Slide 7 but just focus first on Foundations)
- Slides currently focus on the first time setup, need to consider how this is more continuous
- I haven’t talked about the review + approve process or what is human - what is Agent, mine mainly reads as this is humans that need to setup this stuff first (but it may be that can be more explicit that can use agents for this work or to do the leg work etc)

JACK - Intro - Product Foundations

- The core responsibility of product management in SDLC doesn't change with the introduction of AI Agents

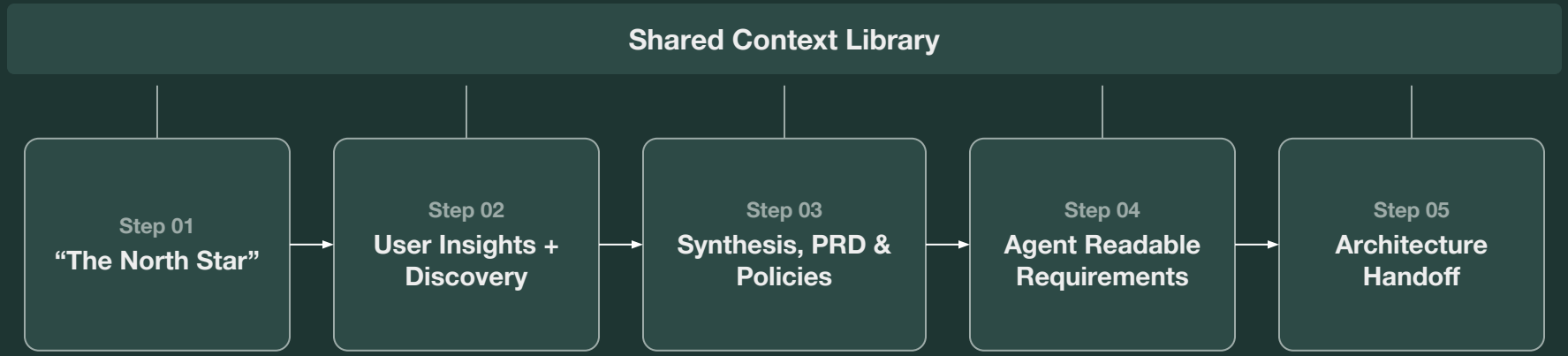
- However with coding agents now able to generate and ship working software in a matter of hours, there is a drastic shift that defending the product's purpose - the "What" and the "Why" - is no longer a preliminary phase - it is the most critical driver of success

- Without a clearly defined product purpose and vision - the sheer speed of agents becomes a liability

- Introduces risks of building contradictory features, duplicating work and/or overwhelming users with unrequested functionality

- Add bullet above doing just enough, keeping lean rather than turning into waterfall

Product Foundations



Establishing the foundations of “What” and “Why”

Step 1: The “North Star” Problem Definition & Strategic Alignment

Defining the core strategic intent and boundaries before technical exploration begins.

What needs to be created

A Business Analysis (BA) document or a Problem Definition brief.

What needs to be agreed

- Scope & specific problem
- Target audience
- Measurable success criteria
- Explicitly *what not to build*

Why it informs context

Acts as the foundational anchor (e.g., `BA-{PROJECT}.md`) that prevents product pivot during high-speed agent code generation.

Step 2 : User Insights & Discovery

Gathering qualitative and quantitative evidence to ensure the product is grounded in reality rather than assumption.

What needs to be created

- Personas
- User Journeys
- Structured Insights (Extracted from transcripts + interviews)

What needs to be agreed

- Core user flows
- Pain Points
- Specific behaviours for target audience

Why it informs context

Personas and journey maps provide context that AI agents will later use to run automated testing at scale, validate the UX, and simulate real-world usage.

Step 3 : Synthesis, PRD and Policies

Synthesis, PRD, and Constraint Mapping Insights are consolidated into a formal specification that aligns both business and user needs while setting strict boundaries.

What needs to be created

- Product Requirements Document (PRD) or initiative brief
- Explicitly documented policies or constraints

What needs to be agreed

- High-level capabilities
- Non-functional requirements (NFRs)
- Security Guidelines
- Regulatory compliance rules
- Platform limitations

Why it informs context

Constraints must be defined early so they can be injected into the central memory file (e.g., CLAUDE.md). If policies are ignored here, they will be missed by the architecture and coding agents, leading to major rework.

Step 4 : Agent Readable Requirement Generation

In an agentic SDLC, traditional requirement documents must be translated into formats that AI agents can perfectly comprehend and execute against.

What needs to be created

An initial requirements backlog consisting of Epics and User Stories using Behaviour Driven Development Format (BDD)

What needs to be agreed

The specific acceptance criteria and testable conditions for every feature.

Why it informs context

BDD acts as the "new programming language" for communicating with agents. These structured feature files (e.g., FEATURE-{EPIC}-{NNN}.md) feed directly into the agent's implementation plan and serve as the baseline for automated test generation

Step 5 : Architecture Handoff

The final step of the Product Purpose phase is consolidating all findings into a structured handoff that explicitly bridges the gap between Product/Requirements and Architecture

What needs to be created

An Architect Handoff Document (e.g., architect-handoff.md)

What needs to be agreed

A cohesive summary of all epics, features, constraints, NFRs, and open questions in one single place.

Why it informs context

Without this handoff, the Architect (whether human or AI agent) would have to piece together context from dozens of separate feature files. This handoff document is the direct input the Architect uses to make technical decisions, draft Architecture Decision Records (ADRs), and create the final plan-context.md that guides the coding agents.

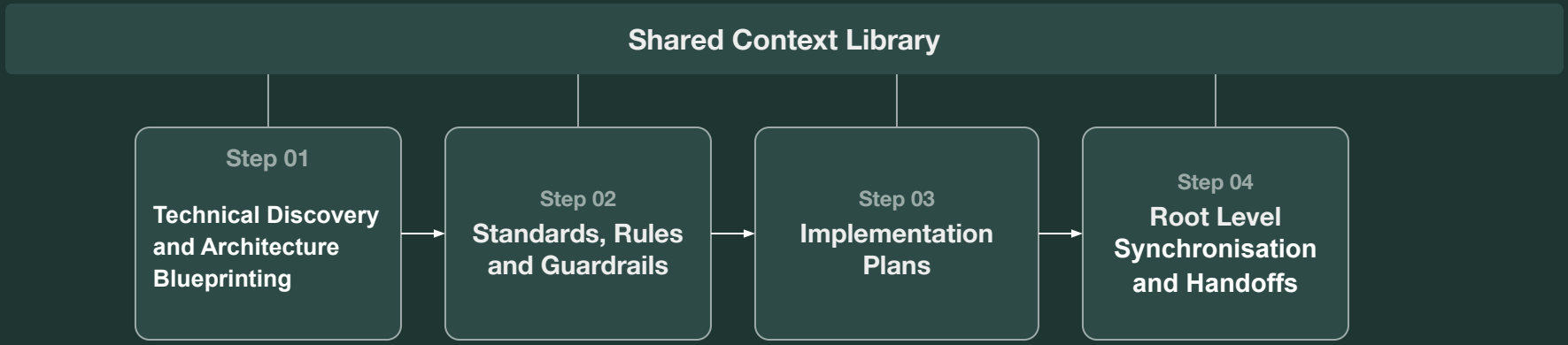
JACK - ARCHITECTURE FOUNDATIONS

In an Agentic SDLC, the human engineering role evolves from managing developers to **architecting ecosystems.** When agents are performing the bulk of the coding, testing, and deployment, the value lies in defining the **guardrails, patterns, and integration logic** that keep the system coherent.

Key Focus Areas

- **Prompt Architecture & Tooling:** Designing the "interfaces" through which agents interact with the codebase, ensuring they have the right tools (APIs, documentation, and sandboxes) to succeed without causing regressions.
- **System Integrity & Governance:** Setting the "Rules of Engagement." This involves defining the CI/CD gates that catch agent-generated hallucinations and ensuring the generated architecture adheres to long-term scalability and security standards.
- **Resolution of Ambiguity:** While agents excel at execution, they often struggle with high-level trade-offs (e.g., *Consistency vs. Availability*). The Tech Lead steps in to make the "human-in-the-loop" decisions that require deep contextual judgment.
- **Code Review at Scale:** Shifting from line-by-line syntax checks to **architectural validation.** The Lead ensures that as agents ship features at 10x speed, the underlying technical debt doesn't grow at the same rate.

Architecture Foundations



TBC

Step 1: Technical Discovery and Blueprinting

The phase begins by taking the architect-handoff.md file from the Product Purpose phase and translating its requirements and constraints into a formal technical design.

What needs to be created

Specific Architecture Decision Records (ADR-{NNN}-{slug}.md) and a master architecture document, typically using a single-file format like arc42.md

What needs to be agreed

- Technical Stack
- Database Design
- API Structures
- Cross-Cutting Concepts
- Platform integrations (Mapping with Systems or API's)

Why it informs context

These documents act as the long-term architectural reference that survives individual iterations, ensuring that as agents build the system, they do not violate past patterns or make conflicting design choices.

Step 2: Establishing Standards, Rules and GuardRails

Before agents write any code, explicit boundaries and quality expectations must be codified into the project's memory.

What needs to be created

•A structured standards/ folder categorized by collaboration, quality, and simplicity.
•Specific "how-to" guides, debugging hints, and examples.
•Additionally, Continuous Integration/Continuous Deployment (CI/CD) pipelines, pre-commit hooks, and secure sandboxed environments (like Docker containers) must be set up

What needs to be agreed

- Strict exit criteria (e.g., demanding 100% unit test coverage),
- Security boundaries
- The specific behaviour-driven development (BDD) or test-driven development (TDD) loop the agent must follow.

Why it informs context

AIs value highly specific instructions and "how-to" guides; providing these standards drastically reduces the time agents take in subsequent iterations and forces them to run security checks and tests before pushing any broken code

Step 3: Drafting the Implementation Plan

The Architect translates the blueprint and standards into a consolidated, actionable plan for the coding agents.

What needs to be created

- The plan-context.md file

What needs to be agreed

- A cohesive summary of all ADRs tech stack decisions, task breakdowns, conventions, and specific exit criteria (such as mandating that a one-line quality test must run green before the plan is considered complete).

Why it informs context

This file serves as the single entry point for the Coder agent. The coding agent loads this plan first and uses it to execute the build, writing back to the plan-context.md file when reality diverges from the original plan

Step 4: Root-Level Synchronisation and Handoff

The final step ensures that the agent's "brain" is loaded with the correct parameters to actually begin development

What needs to be created

The root-level CLAUDE.md (or MEMORY.md) and the phase handoff log (30_handoffs.md)

What needs to be agreed

That the architecture phase is complete and development is officially ready to begin

Why it triggers development

CLAUDE.md acts as the institutional memory that every agent loads at the start of every session to understand project context, conventions, and architecture. Logging the transition in 30_handoffs.md creates a permanent timeline of decisions, allowing the coding agent to trace exactly how the product evolved from the Requirements Engineer to the Architect, and finally to them.

JACK -

PLACEHOLDER - Engineering Foundations

PLACEHOLDER - Data Foundations

Need engineering to also review the Arch Foundations (Split what goes into Arch and what goes into Engineering)

PLACEHOLDER - Add WoW foundations

JACK - Summary of the Agentic Team Structure

Product Lead → Defines the **Destination** (Value/Intent).

Delivery Lead → Manages the **Flight Path** (Flow/Process).

Engineering Lead → Designs the **Aircraft** (Architecture/Boundaries).

Engineers Flies the Plane (Implementation/Validation).

Agent Engineers → The **Engine** (Generation/Velocity).

JACK -

Summary of the Agentic Team Structure

JACK - Summary of the Agentic Team Structure

Role	Responsibility
Product Lead	Defines the **Destination** (Value/Intent).
Delivery Lead	Manages the **Flight Path** (Flow/Process).
Engineering Lead	Designs the **Aircraft** (Architecture/Boundaries).
Engineers	**Flies the Plane** (Implementation/Validation).
Agent Engineers	The **Engine** (Generation/Velocity).

Delivery Lead

The Delivery Lead moves from facilitating people to overseeing the velocity and health of the machine-human interface.

- Workflow Orchestration: Orchestrating the "Agentic Conveyor" to ensure human-to-agent handoffs don't become bottlenecks.
- Break Point Management: Identifies where human intervention is mandatory and ensures these reviews occur quickly to maintain agent productivity.
- Non-Deterministic Risk Management: Proactively identifies risks unique to AI, such as "Context Drift" or agents getting stuck in logic loops, and facilitates human resolution.
- Quality Gatekeeping: Monitors the Acceptance Rate (how often humans approve agent output) and supports interventions if agents start "hallucinating" or drifting from the original requirements.
- Stakeholder "Translator": Bridges the gap between technical agentic metrics (like token efficiency or reasoning stability) and business success criteria (velocity, quality, and ROI).
- Success Metrics: Reports on "Outcome Velocity" and "Time-to-Value" to show how AI acceleration hits business goals.

ROLES - Product

- Intent Definition: Crafts the vision and success metrics with enough precision for agent interpretation without constant clarification.

- Strategic Context: Owns the "Knowledge Base," ensuring it reflects the latest product strategy and user feedback.

- Boundary Governance: Defines the business and ethical "No-Go" zones, such as unauthorized pricing changes or data access.

- The Final Value Gate: Acts as the ultimate sign-off for agent-generated features to ensure they solve the human user's problem.

- Continuous Value Validation: Since agents accelerate delivery, the PM focuses on "Rapid Prototyping Loops," using agent speed to test hypotheses with users in days rather than months.

- Feedback Loop Optimization: Aggregates user feedback and translates back into the shared context to ensure the entire agent swarm learns from every release.

ROLES - Engineering lead or Architect

The Engineering lead or Architect transitions from a code reviewer to a System Architect and Guardrail Engineer.

- Architectural Boundaries: Defines the "Sandbox" (tech stack, design patterns) where agents operate.
- Structural Integrity: Enforces zones where agents cannot alter core logic without human oversight.
- Security & Compliance Guardrails: Sets the "Secure-by-Design" principles. They ensure agents aren't introducing vulnerabilities or using deprecated, "hallucinated," or insecure libraries.
- Sprawl Governance: Monitors the codebase for "Agent-generated Sprawl" to prevent unmaintainable "spaghetti AI"
- System Health Standards: Establishes the telemetry and logging standards that agents must optimize against
- Review Orchestration: Configures "Reviewer Agents" (linters, AI-reviewers) as the first line of defense

ROLES - Engineers

The Human Engineer transitions from a "manual laborer of code" to an Agent Editor and Logic Specialist.

- Collaborative Implementation: Works side-by-side with agents, personally focusing on high-complexity logic and creative problem-solving
- The Final Integrity Gate: Performs high-level architectural and security reviews on agent-generated PRs, checking for "hallucinated" dependencies
- Technical Context Curator: Maintains the "Long-term Memory" of agents by updating ADRs, READMEs, and technical documentation
- Agent Coaching: Refines agent prompts or context after bugs to ensure the agent doesn't repeat mistakes
- Logic Specialization: Manually handles edge cases and legacy integrations that agents struggle to execute reliably

The Agentic Engineer is a digital entity assigned to discrete, repeatable technical tasks.

- Discrete Skill Execution: Operates within a specific persona (e.g., Code Generation or Unit Test Specialist) to prevent "reasoning drift"
- Autonomous Implementation: Writes underlying code based on high-fidelity "Intent" while adhering to Engineering Lead -defined boundaries
- Continuous Self-Correction: Analyzes build logs to self-heal code before alerting the human team
- Metadata Production: Automatically generates technical documentation and PR summaries explaining the "Why" behind changes
- Automated Testing: Drafts unit and integration tests simultaneously with code production.

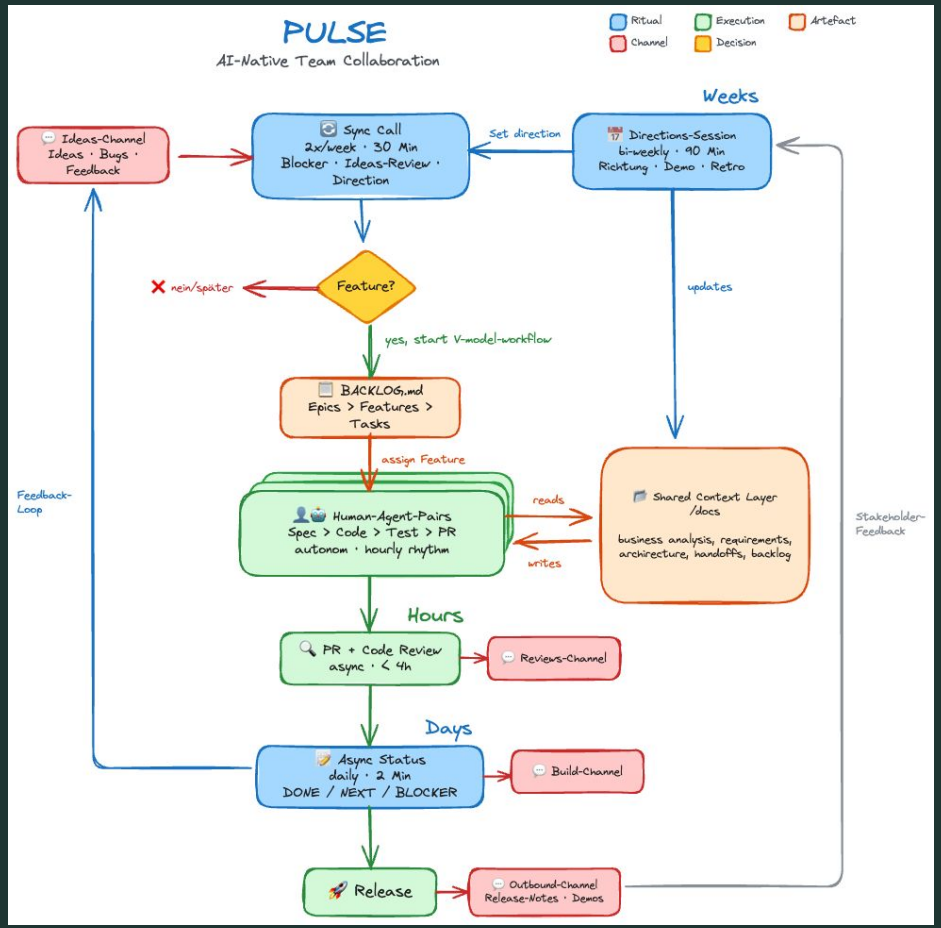
Types of Agentic Engineers:

You may decide to have multiple agents in the Agentic SDLC cycle, each with defined skills. Agreeing on the skillset + defining the build of each persona should be done as a team when kicking off the project and reviewed at regular periods as you inspect + adapt. Cost/budget should also be considered

Skill Profile	Key Responsibility	Primary Toolset / Context
The Architect Agent	Translates PM Intent into technical **Specs**.	System Design Docs, API Schemas.
The Coder Agent	**Writes code** and logic based on the Spec.	IDE, Repo access, Library docs.
The Tester Agent	Generates **Unit & Integration Tests**.	Test Frameworks, Mock Data.
The Reviewer Agent	Performs **Code Reviews** and linting.	Security Guardrails, Style Guides.
The Refactor Agent	Targets **Technical Debt** and optimization.	Profiling tools, Best Practice guides.

The Differences

Event	Scrum	Kanban	Agentic
Cadence	Regular time-boxed Sprints (1–4 weeks).	Continuous flow (no fixed cycle).	Real-time / Asynchronous "Heartbeats."
Planning	**Sprint Planning**: Big upfront batching.	**Just-in-Time**: Pull-based planning.	**Sync**: Dynamic, AI-assisted triage.
Status Update	**Daily Stand-up**: Fixed 15-min sync.	**Optional**: Focus on board bottlenecks.	**Automated**: A-sync AI-generated status updates
Review	**Sprint Review**: Demo at end of sprint.	**Milestone-based**: Occurs as needed.	**Continuous Delivery**: Reviews are instant.
Retrospective	**Sprint Retrospective**: Scheduled reflection.	**Kaizen**: Continuous improvement.	**Instant Feedback**: Loop-based learning.


PULSE
AI-Native Team Collaboration
Ritual
Execution
Artefact
Channel
Decision
Weeks
Ideas-Channel
Ideas · Bugs · Feedback
Sync Call
2x/week · 30 Min
Blocker · Ideas-Review · Direction
Set direction
Directions-Session
bi-weekly · 90 Min
Richtung · Demo · Retro
nein/später
Feature?
updates
yes, start V-model-workflow
BACKLOG.md
Epics > Features > Tasks
assign Feature
Feedback-Loop
Human-Agent-Pairs
Spec > Code > Test > PR
autonom · hourly rhythm
reads
writes
Shared Context Layer
/docs
business analysis, requirements, architecture, handoffs, backlog
Stakeholder-Feedback
Hours
PR + Code Review
async · < 4h
Reviews-Channel
Days
Async Status
daily · 2 Min
DONE / NEXT / BLOCKER
Build-Channel
Release
Outbound-Channel
Release-Notes · Demos

Ceremony	Frequency	Objective	Key Participants
Daily Stand Up			
Spec Review (Three Amigos)	Per Epic?	Review the **Technical Specification** and **Requirements Definition** before Implementation begins.	Product, Delivery and Engineering
Feedback Loop (Retro)	Every 2-4 Weeks	● Analyse and review performance metrics ● What did production data tell us? ● How can we improve the next cycle?	Full Delivery Team

Measuring Success (The "Agentic" Metrics)

Traditional velocity (story points) is less relevant when an agent can write 1,000 lines of code in seconds. Focus on **Flow and Quality**:

- **Lead Time to Live:** Time from Product "Intent" to Code in Production.
- **Agent Autonomy Rate:** % of PRs merged without human code intervention. (TBC)
- **Bug Leakage:** How many defects bypass the QA Agent? (This measures the efficacy of your agent's testing suite).
- **Token Efficiency vs. Output:** Monitoring cost-to-delivery to ensure the agentic flow remains ROI-positive.

Chirag – Agentic SDLC

Delivery – Requirements Definition

1. Discovery and Information Gathering (The Inputs) –The process begins with "Discovery Interviews" and stakeholder alignment. The goal is to capture raw intent before it is filtered.

Action: Humans conduct interviews or workshops.
Agentic Support: Agents transcribe and extract insights from these sessions.
Inputs: Meeting transcripts, initiative briefs, and target outcomes.

2. Synthesis and Persona Mapping (The Shared Context) – Raw data is converted into structured Shared Context. This makes the requirements "explicit, versioned, and accessible to both humans and AI".

Action: The Agent organizes data into User Personas, User Journeys, and Design Systems.
Artefacts: personas.md, user-journeys.md, and target-outcomes.md are added to the repository.

3. Collaboration and Writing (The Governance) – This is where the requirements are formally drafted. Using Agent Skills, the AI helps format the requirements into a consistent structure, such as BDD.

Execution: The Agent synthesizes the gathered insights into a "PRD" or "Initiative Brief".
Output: The primary output is a set of Requirements/Stories and, if applicable, a Prototype Design.

4. The Expert-in-the-Loop Review (The Human Element) – Requirements are never "Final" until an expert validates them.

Action: Product and Design teams review the synthesized stories and prototypes.
User Testing: If necessary, a "User Test" is performed on prototypes to ensure the requirements solve the right problem.

5. Transition to "Ready" (The Outcome)

The stage concludes when the requirement is marked as "Refined Feature Ready".
This acts as the signal for the agent to begin Technical Spec Creation.

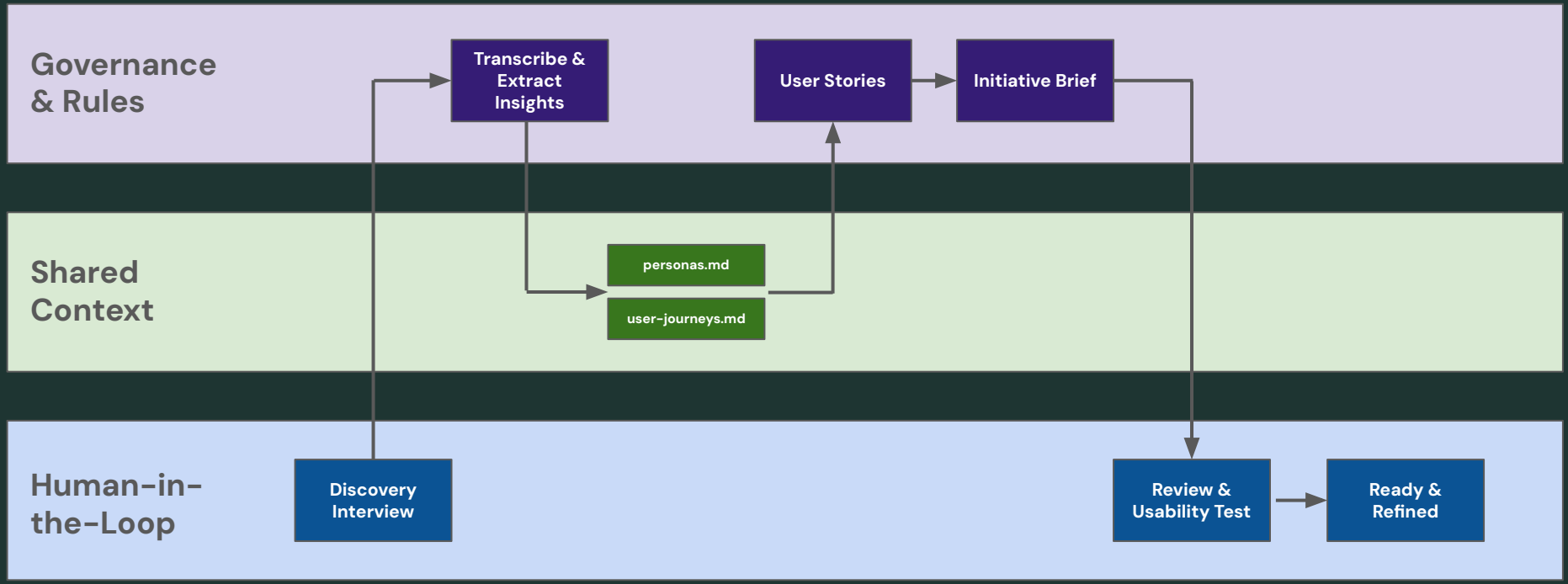
Inputs

- **Stakeholder Intent:** High-level goals, business value, and the "why" behind the feature.
- **Discovery Evidence:** Meeting transcripts, interview notes, or Slack discussions where requirements were initially debated.
- **Design Assets:** Low-fidelity wireframes, UX prototypes, or existing design system documentation.
- **Foundation Context:** Existing architecture diagrams and product domain rules that the new requirements must respect.

Outputs

- **Refined User Stories (BDD):** Feature descriptions written in a "Given/When/Then" format to remove ambiguity for the AI.
- **Persona & Journey Maps:** personas.md and user-journeys.md files that ground the AI in the user's perspective.
- **Product Requirements Document (PRD):** A synthesized initiative-brief.md that captures target outcomes and success metrics.
- **Requirement ID Metadata:** A unique identifier (e.g., REQ-101) that links these docs to the later technical specs and code commits.

Delivery – Requirements Definition



Chirag - Agentic SDLC - Step 2 - Technical Specification

1. Triggering the Stage (Human-led)

Action: The Human confirms the feature is "Ready" in the project management tool (e.g., moving a GitHub Issue to a specific column).
Why: AI should not start technical work until a human has validated that the product intent (Stage 00) is clear.

2. Context Gathering (Agent-led)

Action: The Agent consumes the Shared Context by "reading" the repository.
Execution: It specifically scans for architecture.md, service-contracts.md, and any existing ADRs (Architecture Decision Records).
Goal: The Agent builds a mental map of the system's current boundaries so it doesn't suggest a solution that breaks existing rules.

3. Drafting the Blueprint (Agent-led)

Action: The Agent consumes the Governance & Rules (System Instructions/Skills) to write the spec.md.
Execution: Using the product intent and the technical context it just gathered, the agent generates a draft that includes:
- Proposed file changes and new file paths.
- API contract updates.
- A step-by-step implementation plan for Stage 02.

4. Steering and Refinement (Human-led)

Action: The Human reviews the draft spec.md and provides feedback via comments (e.g., in a Slack thread or a Pull Request).
Execution: The Human might say, "We shouldn't use a new database table for this; use the existing 'metadata' column instead."
Goal: To apply "Expert Judgment" on trade-offs that an AI might not prioritize (e.g., cost, long-term maintenance, or team conventions).

5. Final Approval (Human-led)

Action: The Human explicitly "Approves" the spec.
Outcome: This approval acts as the Governance Gate. The Agent is now authorized to move to Stage 02: Implementation & Code Gen.

Inputs

- Refined Features / User Stories - the primary trigger
- Product Context - PRDs, initiative briefs, target outcomes
- Design Artefacts - Prototypes and UX wireframes
- Transcripts / Meeting Notes - Raw context from discovery interviews, etc

Outputs

- Technical Specification (spec.md) - comprehensive document, e.g. paths, tasks, integrations
- Acceptance Criteria / Tests - explicit validation points
- Update to Shared Context - a draft of how the system architecture or interfaces change

Delivery – Technical Specification

1. Ingesting Requirements (The Trigger) - The process is triggered once a feature is marked "Refined Feature Ready" in the previous stage.

Action: The AI Agent reads the newly refined user stories and product intent from the repository.
Expert-in-the-Loop: A human ensures that the business requirements are sufficiently detailed before the agent begins architectural mapping.

2. Grounding in Shared Context (The Guardrails) - The Agent must align the new feature with the existing system.

Execution: The Agent consults the "First-Class Assets" in the repository, specifically architecture.md, service-contracts.md, and coding-standards.md.
Goal: To ensure the proposed solution doesn't violate existing technical constraints or architectural integrity.

3. Drafting the Technical Blueprint (The Skill) - Using its specialized Spec Creation Skill, the Agent drafts the spec.md.

Action: The Agent applies "System Design Rules" to map requirements to specific implementation tasks.
Content: The draft includes API contracts, database schema changes, file paths, and integration points.

4. Expert-in-the-Loop Review (The Governance Gate) - This is a critical "hard gate" where humans steer the outcome.

Action: A human expert reviews the spec.md for trade-offs, security, and complexity.
Steering: The expert provides feedback to refine the plan, ensuring the AI isn't "hallucinating" a sub-optimal technical path.

5. Approval and Hand-off (The Outcome) - The stage concludes with the "Approve Spec" action.

Outcome: The approved spec becomes the "contract" for the Developer Agent in the implementation stage.
Context Update: The draft spec is now a living artifact that will eventually be merged into the permanent Shared Context.

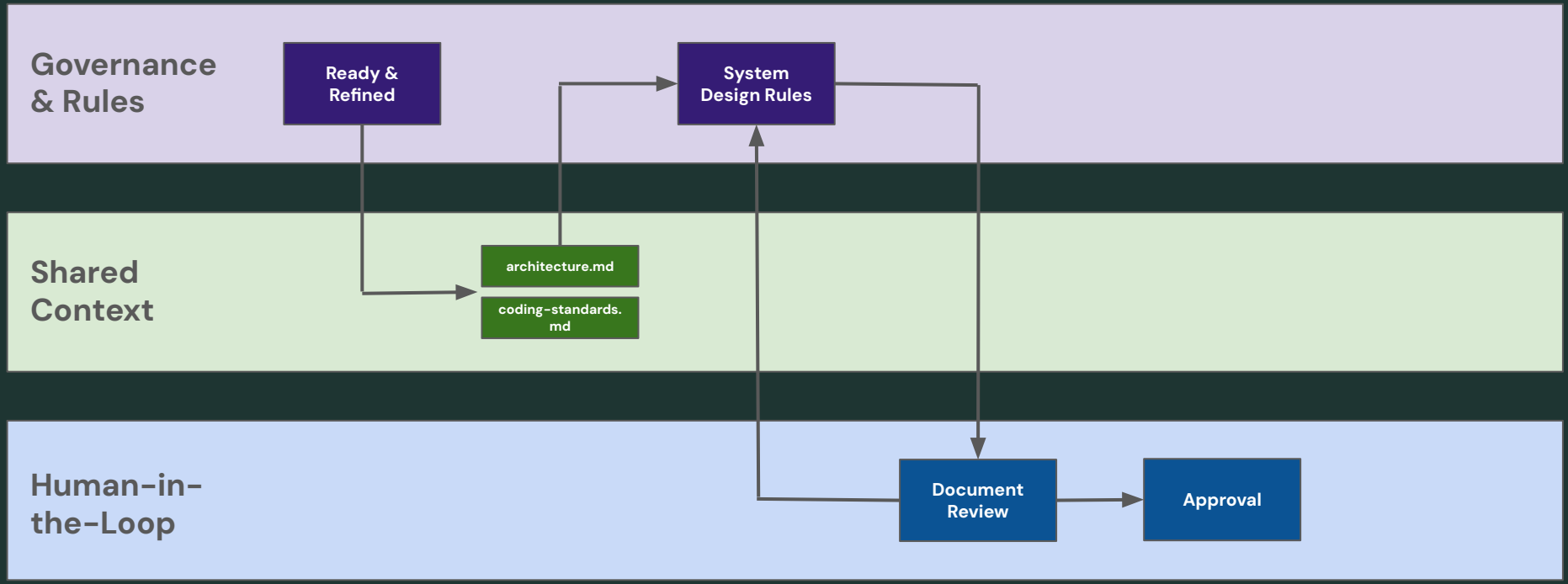
Inputs

- **Refined User Stories:** The BDD-formatted requirements from requirements stage (e.g., feature_auth.md) that have been marked as "Refined Feature Ready".
- **Architecture & Design Artifacts:** Existing documentation like architecture.md, system diagrams, and UI/UX prototypes.
- **Service & Data Contracts:** Current API definitions and database schemas to ensure integration compatibility.
- **Coding Standards:** Explicit rules and templates (e.g., coding-standard.md) that the agent must follow for consistency.

Outputs

- **Technical Specification (spec.md):** A comprehensive document detailing file paths, implementation tasks, and architectural decisions.
- **Implementation Plan:** A step-by-step breakdown of atomic coding tasks for the agent to execute in the implementation stage.
- **Draft Acceptance Tests:** Initial test cases and edge-case analysis derived from the spec to be used during validation.
- **Updated Shared Context:** Modifications or updates to existing technical documentation (like ADRs) reflecting the new technical approach.

Delivery – Requirements Definition



Delivery – Implementation

1. Context Loading (The Trigger) - The process begins when the agent receives the "Approve Spec" signal from the previous stages.

Action: The Agent loads the specific Technical Specification (spec.md) and the relevant Shared Context (existing code, service contracts, and coding standards).
Goal: To ensure the AI is strictly "grounded" in the pre-approved plan before it begins generating code.

2. Atomic Code Generation (The Execution) - The Agent executes the tasks defined in the implementation plan.

Action: Using the "Code Gen" skill, the AI generates code in small, atomic chunks rather than one giant block.
Constraint: The Agent must strictly adhere to the Governance & Rules, applying "Pattern Enforcement" and "Code Quality" standards encoded in your system.

3. Continuous Validation (The Inner Loop) - As code is generated, it must be validated against the technical constraints.

Action: The Agent (or a secondary CI agent) runs "CI Validation" checks to ensure the new code doesn't break existing builds or violate architecture.
Feedback: If a check fails, the Agent self-corrects based on the error logs without needing human intervention at every step.

4. The Expert-in-the-Loop Review (The Quality Gate) - Once the code is complete, it enters a formal human review phase.

Action: A human expert performs a "Review PR" (Pull Request).
Steering: The human looks for nuances the AI might miss—performance edge cases, security vulnerabilities, or alignment with long-term maintenance goals.

5. Approval and Merge (The Outcome) - The stage concludes with the "Approve Merge" signal.

Outcome: The code is merged into the codebase.
Note: In this model, the feature is not "Done" yet; it still needs to pass the Validation stage and Context Update phases.

Inputs

- **Approved Technical Specification (spec.md):** The primary source of truth from previous stage containing the task breakdown and API contracts.
- **Target Repository Context:** Access to existing code files, interfaces, and dependencies that the new feature will interact with.
- **Governance & Rulesets:** Specifically the Pattern Enforcement and Code Quality rules (e.g., lint rules, Prettier configs, or specific .cursorrules).
- **Data Contracts & Schemas:** Definitions for any database migrations or external API responses the code must satisfy.

Outputs

- **Feature Branch / Source Code:** The actual logic, UI components, and infrastructure-as-code files generated by the agent.
- **Atomic Commits:** A series of logical, small code changes that are easy for a human to review.
- **Pull Request (PR) Description:** An agent-generated summary of the changes, explaining how the implementation satisfies the spec.md.
- **CI Validation Reports:** Initial feedback from automated checks (linting, basic unit tests, build status) that the agent must pass before the PR is "ready".

Delivery – Validation & Testing

1. Loading the Truth (The Trigger) - The process starts once the code is merged in previous stage

Action: The Agent loads the User Stories/BDD (from requirements stage) and the Technical Spec (from technical spec stage).
Goal: To ensure the test suite is verifying the intent (the "why") and the contract (the "how"), not just the code itself.

2. Test Suite Generation (The Execution) – The Agent creates the automated validation layer.

Action: Using a "Test Generation" skill, the AI generates unit, integration, and end-to-end tests based on the acceptance criteria defined in the requirements.
Edge Case Analysis: The AI is specifically tasked with performing "Edge Case Analysis"—identifying potential failures that a human might overlook.

3. The Regression Run (The Check)

Action: The Agent executes a "Regression Suite Run".
Constraint: The code must pass not only the new tests but also verify that no existing functionality was broken by the merge.

4. The Expert-in-the-Loop Review (The Quality Gate)

Action: A human expert reviews the Test Results.
Steering: The expert looks at the test coverage and the quality of the edge cases. They ask: "Does this suite truly give us the confidence to deploy?".

5. Approval (The Outcome)

Action: The human provides the "Approve Suite" signal.
Result: The feature is now functionally verified and ready for the final step: updating the permanent project context.

Inputs

- **User Stories & BDD Scenarios:** The "Given/When/Then" flows from requirements stage that define the expected behavior.
- **Technical Specification (spec.md):** The blueprint from the previous stage that defines the API contracts, data schemas, and edge cases to be tested.
- **The Implemented Code:** The actual merged source code from implementation stage that needs to be exercised.
- **Existing Test Suite:** The current codebase of tests to ensure the Regression Suite Run covers the whole system.

Outputs

- **Automated Test Suite:** New unit, integration, and E2E tests (often in Markdown or code-native formats) that are now part of the repository.
- **Edge Case Analysis Report:** A summary of the "stress tests" the AI performed to find silent failures.
- **Validation Results:** A report showing the passing/failing status of the new and existing tests.
- **Test-Generated Artifacts:** Mock data or environment configurations used during the validation run.

Delivery – Deployment

1. Signal Verification (The Trigger) - Deployment is triggered only when the "Approve Suite" signal is received from previous stage.

Action: The Deployment Agent verifies that all unit, integration, and E2E tests have passed and that the "Expert-in-the-Loop" has signed off on the results.
Goal: To ensure no code reaches production without explicit functional validation.

2. Environment Readiness & Pre-Flight (The Guardrails) - Before the code moves, the agent checks the health of the target environment.

Action: The Agent runs "Infrastructure-as-Code" checks and verifies environment variables against the Governance & Rules layer.
Constraint: If the environment is unstable or a configuration drift is detected, the agent halts the process and alerts a human.

3. Automated Rollout (The Execution) - The Agent orchestrates the delivery of the code.

Action: Depending on the "Skill" defined, the agent performs a Blue/Green or Canary deployment.
Real-time Monitoring: The Agent monitors production telemetry (errors, latency) in real-time. If anomalies are detected, the agent initiates an automated rollback based on predefined "Rules".

4. The Expert-in-the-Loop Smoke Test (The Quality Gate) - Even in an automated rollout, a final human "Expert" check ensures the system feels right.

Action: A human expert performs a quick "Smoke Test" or reviews the post-deployment dashboard.
Sign-off: The human provides the "Deploy Verified" signal.

5. Hand-off to Context Lifecycle (The Outcome)

Action: Once verified, the deployment agent signals the Context Lifecycle stage to update the documentation to reflect that the feature is now "Live".

Inputs

- **Approved Test Suite Results:** The final pass report from Stage 03, including edge case analysis.
- **Target Environment Configuration:** Infrastructure-as-Code (IaC) files or environment secrets required for the specific destination (Staging, UAT, or Prod).
- **Deployment Skill/Rules:** Explicit instructions defining the rollout strategy (e.g., "Only deploy if latency is <200ms" or "Use Blue/Green swap").
- **Expert "Go" Signal:** A specific human sign-off that the business is ready for the change to go live.

Outputs

- **Deployed Artifacts:** The running containers, binaries, or serverless functions in the production environment.
- **Deployment Logs & Telemetry:** Real-time data showing the health of the rollout, including error rates and system performance.
- **Smoke Test Report:** A summary of the post-deployment "Expert" check to confirm the feature is behaving as expected in the wild.
- **Rollback Records (if applicable):** Documentation of any automated or manual reverts performed during the process.

Delivery – Customer Feedback

1. Signal Collection (The Trigger) – The process begins once the feature has been live for a defined period or has reached a specific usage threshold.

Action: The Feedback Agent aggregates data from multiple sources: user analytics, support tickets, app store reviews, and direct NPS surveys.
Goal: To move beyond anecdotal evidence and gather a statistically significant view of how the feature is being used.

2. Sentiment & Gap Analysis (The Execution) – The Agent analyzes the collected feedback against the original intent defined in Stage 00.

Action: Using "Natural Language Processing," the AI categorizes feedback into "Bugs," "Feature Requests," or "UX Friction".
Insight: The Agent identifies the gap between what we expected users to do (from requirements stage user journeys) and what they are actually doing.

3. Contextual Synthesis (The Memory) – The feedback isn't just stored in a dashboard; it is fed back into the project's permanent record.

Action: The Agent drafts updates to the user-journeys.md and target-outcomes.md in the /docs folder.
Result: The "Shared Context" now reflects the real-world performance of the system, preventing the AI from making the same assumptions in future iterations.

4. The Expert-in-the-Loop Triage (The Quality Gate) – Humans decide which feedback warrants a new turn of the SDLC circle.

Action: Product Managers and Engineers review the synthesized feedback report.
Decision: The team decides whether to "Close" the feature as successful or trigger a new "Requirements Definition" to address discovered gaps.

5. Restarting the Circle (The Outcome)

Action: Approved feedback items are converted into new "Refined Features".
Outcome: The SDLC circle begins again, fueled by evidence rather than assumptions.

Inputs

– Usage Analytics: Data on how users are actually navigating the feature (e.g., click-stream data, drop-off rates).
– Direct Feedback: Support tickets, NPS scores, app store reviews, or Slack/Intercom conversations.
– Original Target Outcomes: The target-outcomes.md from requirements stage, used as a benchmark to see if the feature met its original goals.
– Production Telemetry: Error rates or performance metrics that might indicate "silent" user frustration.

Outputs

– Synthesized Feedback Report: A categorized summary of what is working and what isn't, mapped back to specific feature IDs.
– Updated User Journeys: Modifications to user-journeys.md reflecting how users actually behave versus how we assumed they would.
–Contextual "Bugs" or "Gaps": A list of identified discrepancies between intent and reality, added to the project backlog.
– Revised Architecture/Standard Notes: If feedback suggests the tech stack is causing UX friction, these foundations are updated for the next turn.

Delivery – Context Lifecycle

1. Context Diff Analysis (The Trigger) - This stage begins immediately after a feature is verified and deployed. **???**

Action: The Agent performs a "Context Diff Analysis".
Goal: The AI compares the current state of the code with the existing documentation to identify exactly what has changed.

2. Synchronizing Artifacts (The Execution) - The Agent updates the permanent project records to reflect the new reality.

Action: The Agent applies "Documentation Standards" to update living files like architecture.md, interfaces.md, and dependency-map.md.
Rule: A feature is not considered "Complete" until it is both "Code Complete" and "Context Complete".

3. Consistency Check (The Guardrails) - Before the update is finalized, the system ensures the new context is coherent.

Action: The Agent runs a "Consistency Check" across all interconnected markdown files.
Result: This prevents contradictions (e.g., a spec saying an API returns a string while the updated architecture doc says it returns an object).

4. The Expert-in-the-Loop Review (The Quality Gate) - Humans verify that the project's "Source of Truth" remains accurate.

Action: A human reviews the changes to the documentation (the "Context PR").
Sign-off: The human performs the "Verify & Close Ticket" action.

5. Closing the Loop (The Outcome)

Outcome: The updated context becomes the foundation for the next turn of the SDLC circle.
Benefit: This keeps the AI aligned with reality, reducing future hallucinations caused by outdated information.

APPENDIX STUFF

Principles that underpin the Agentic Ways of Working

1. **Human Accountability is Non-Negotiable**
2. **Artefacts are the Source of Truth**
3. **Orchestration and Governance are Architectural**
4. **Bounded Agency with Explicit Escalation Triggers**
5. **Agents Surface Work; Humans Own the Backlog**
6. **Shared Context is the Operating Substrate**
7. **Continuous Validation Against Source Artefacts**
8. **Failure is Explicit, Recoverable, and Never Silent**
9. **Agent Handoff Integrity**
10. **Observability and Traceability by Design**
11. **Resource Envelopes are Mandatory and Enforced**
12. **Security and Isolation by Default**
13. **Incremental Autonomy Maturity**
14. **Controlled Evolution of the Agentic System**

Product Operating Model

Principles that underpin the ways of working

- Outcomes over outputs
- Small, empowered, cross-functional, autonomous teams
- Cross-functional leadership at the different flight levels
- Be agile, stay lean
- Continuous planning at different flight levels
- Leverage data
- Continuous learning and discovery
- Engineering excellence is an enabler for faster value delivery

This playbook defines how work flows from strategy to delivery.

It exists to create **clarity, consistency, and predictability** at scale so leaders can **make informed decisions**, **teams can focus on delivery**, and Hargreaves Lansdown can **move faster without creating hidden risk** or disruption.

What this playbook is NOT

- **Not rigid**: Squads need guardrails and autonomy, not scripts,
- **Not a one-off change** : Planning and improvement is continuous and principle led
- **Not advocating bypassing decision forums and hidden trade-offs**: Agility requires, fast decisions with visible trade-offs

What this playbook aims to be

- **Be practical and scalable:** Enough structure to be adopted widely and scaled
- **Provide clear guardrails** that enables pace and autonomy
- **Drive outcomes orientation:** Focused on value, not activity
- **Drive data maturity:** Leveraging data to promote the right behaviours and drive planning and improvement

Problems the Playbook WoW aims to solve

- No clear entry point for work
- Priorities set without full context
- Dependencies surface too late
- Commitments made too early
- Urgent work breaks flow
- Unclear ownership of decisions
- Planning treated as a one-off
- Weak learning loops

The Operating Rhythm

How work flows from strategy through to delivery

Strategic Layer Planning

3 - 5 Year Strategic Horizon reviewed bi-annually

(Owned at Exec Level sets direction for the organisation)

Annual Strategic Roadmap Review

Monthly Board Level Mission Control

Strategic Outcomes

Coordination Layer - Tribe Level

12 Month Outcomes
reviewed quarterly to align with Strategy

(Owned by Business Lead/Head of Product/SEMs)

Pipeline Review

Tribe Quarterly Planning

Crosstribe Alignment

Monthly Product Showcase

Fortnightly Prod / Engineering

Tribe Outcomes / Opportunities / Initiatives

Operational Layer - Squad Level

6 month roadmap owned by PMs reviewed quarterly

(Delivery Plan owned by squads reviewed fortnightly)

Squad Quarterly Planning

Sprint Planning

Daily Stand Up

Sprint Review

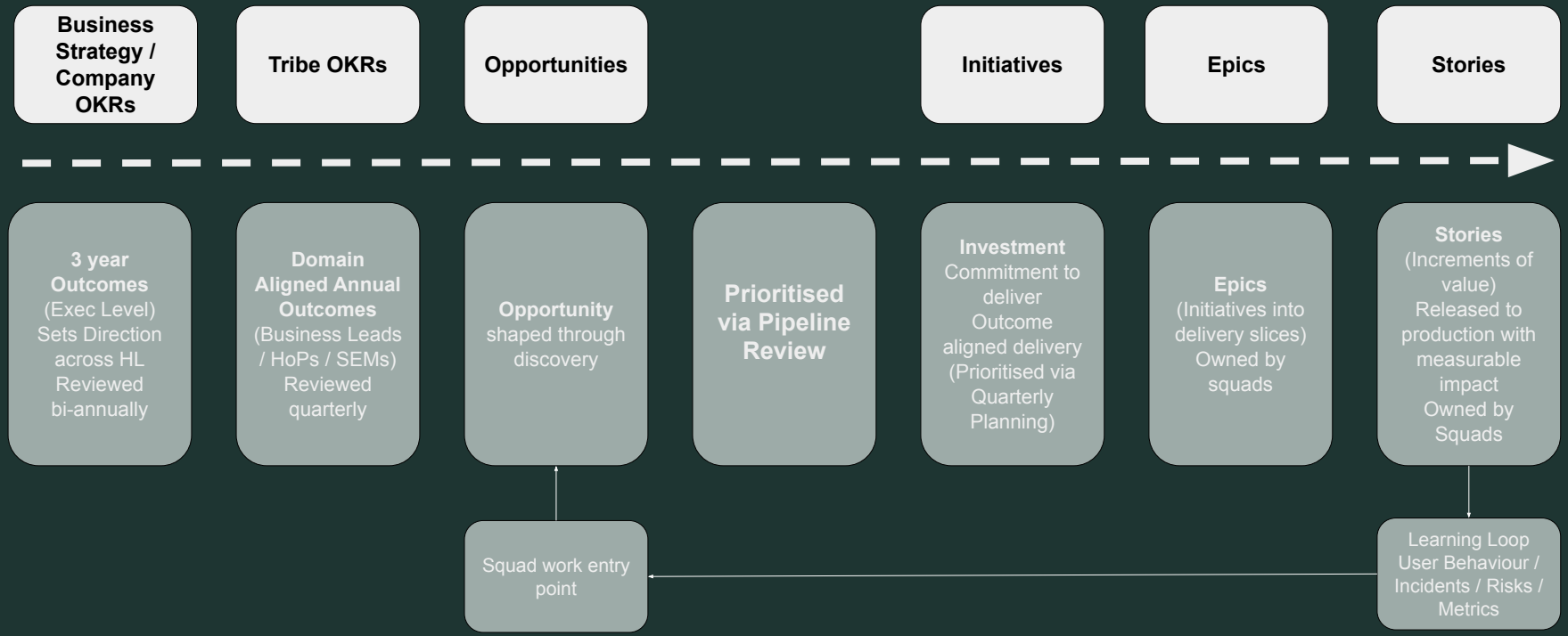
Sprint Retrospective

Epics / Stories

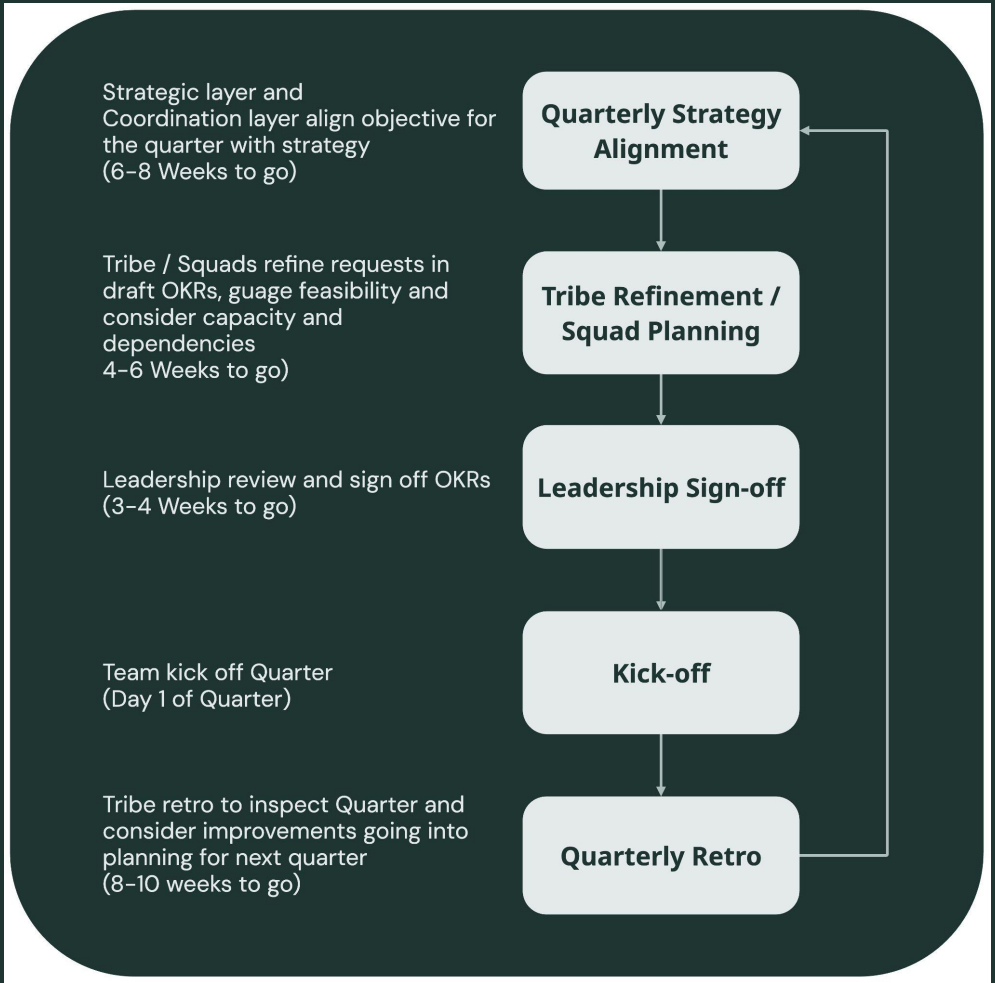
Aligning/Planning

Learning / Reporting

How work flows from strategy through to delivery



Quarterly Planning absorbs prioritised initiatives from Pipeline Review, aligns capacity and dependencies, sets clear direction for the squads for the coming quarter.



Cross-tribe dependency management

With multiple tribes, dependencies must be managed deliberately to enable sequencing, flow, and predictable delivery.

Quarterly Cross-Tribe Dependency Planning

Before the quarter starts:

- Aligns initiative sequencing across tribes
- Surfaces and resolves known dependencies
- Confirms what is feasible within capacity

Outcome:

- Cross-tribe initiative sequence
- Clear dependency ownership
- Known delivery risk areas

Mission Control – Cross-Tribe Dependency Review

During delivery:

- Surfaces active and emerging blockers
- Coordinates resolution across tribes
- Escalates risks early

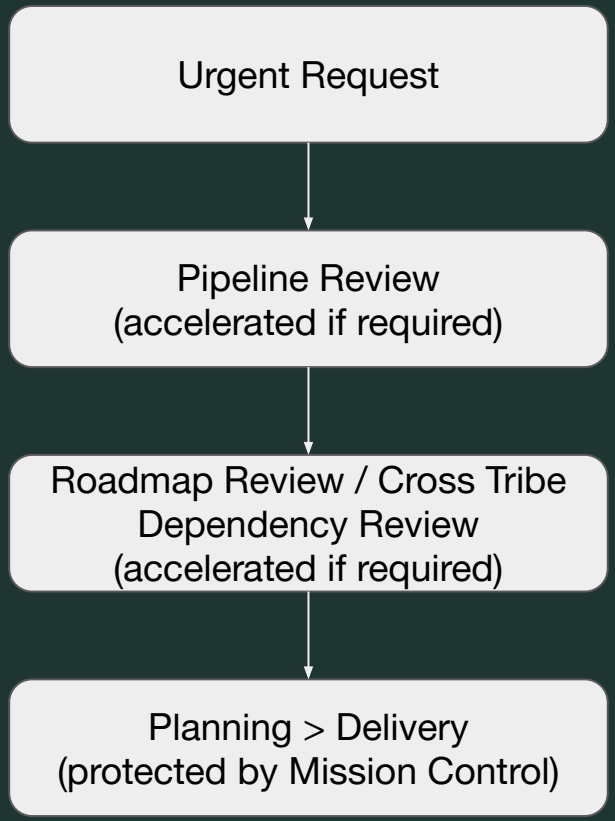
Outcome:

- Blockers resolved or owned
- WIP protected
- Delivery risk actively managed

A shared **Dependency board** will enable live visibility of dependencies to feed into Pipeline Reviews, Mission Control, and Quarterly Planning.

Forum	Purpose	Key Decision	Decision Owner	Involved	Frequency
Strategy Review	Set long-term direction and outcomes	Define and adjust strategic outcomes and priorities (3–5 year horizon)	CCO / CTO	Executive Leadership Team / HoPs / Business Leads	Bi-annual
Pipeline Review	Decide what to invest in and when	Approve, defer, decline, or stop initiatives and discovery; prioritise across the portfolio	Business Lead / HoP	Heads of Product, Senior Engineering Managers	Regular cadence plus ad hoc for urgent requests
Quarterly Planning	Translate pipeline decisions into delivery intent	Prioritise approved initiatives, Sequence, Align with capacity; decide what continues, waits, or stops.	Tribe Leads (with Business Lead)	Product, Engineering Leads	Quarterly
Cross-Tribe Dependency Planning	Resolve dependencies before delivery begins	Agree on cross-tribe sequencing, dependency ownership, and feasibility	Tribe Leads	Heads of Product, Senior Engineering Managers, Platform / Architecture Leads	Quarterly (approximately 4 weeks before the quarter)
Mission Control	Protect the delivery flow once work is in flight	Manage WIP, escalate blockers, decide when work slows, pauses, or is re-sequenced	CCO / CTO	Product and Engineering Leads from each tribe	Weekly or fortnightly

Urgent requests do not bypass the system, they accelerate decisions.



*Urgent requests are typically regulatory / contractual / major incidents too large to be absorbed by squads level

Risks are not absorbed by the team, intentional decisions are made at pipeline review based on all the information and a rebalancing of plans and impact on strategic goals.

The forums that handle urgent requests

- **Pipeline Review**
  Is the request genuinely urgent? Decision to proceed, delay, reduce scope, or decline
- **Tribe / Cross-Tribe Dependency Review**
  Are dependencies resolved so that squads are ready to start?
- **Roadmap Review**
  Makes the impact of urgent work (risks and impact on plans) visible and rebalances plans
- **Mission Control / Delivery Health Review**
  Protects squads with in flight delivery

The role of the Pipeline Review

- **Evaluate shaped opportunities -** Validate problem and size with evidence, ensure outcome alignment and readiness to tackle.
- **Prioritise opportunities using evidence -** Apply RICE or Cost of Delay and outcome alignment to compare value.
- **Approve entry into discovery -** Authorise which opportunities are worth exploring now, later, or not at all.
- **Sequence opportunity flow across squads -** Ensure discovery and technical exploration are balanced with delivery capacity.
- **Surface dependencies early -** Prevent cross-tribe and platform issues from becoming blockers during delivery.
- **Inform initiative shaping -** Building the short and mid-term opportunity outlook will shape future initiatives.
- **Protect predictability -** Prevent random requests, or unvalidated ideas from consuming squad capacity.

Product Discovery enables outcome-led and predictable delivery by ensuring squads invest in work that is grounded in evidence, not assumption.

It is how squads reduce uncertainty, validate problems and test solution direction before committing to delivery.

Squads use Product Discovery to:

- Shape opportunities ahead of Pipeline Review
- Explore solution options after Pipeline Review
- Run near and long-term technical discovery to de-risk delivery

Delivery

How work flows through the teams

	Initiative agreed	Early discovery & feasibility	Approach, sequencing agreed and risks managed	Rolling delivery planning and forecasting	Just in time story shaping	Build, release, learn
What happens	It is agreed that this problem is worth addressing and that investment is to be made for a team to explore and deliver value	Teams explore the problem, possible approaches, and key risks. This may include user research, technical investigation, and dependency discovery	Squads and tribe leadership align on what to tackle first, known dependencies, and areas of uncertainty. The work starts in a sensible order.	Squads create a short-term delivery outlook showing likely progress and confidence levels. Forecasts are updated as learning increases.	Stories are shaped shortly before work starts so they stay small, clear, and relevant. Anyone can suggest work; teams decide what is ready to build.	Teams build and release small changes, observe real-world results, and adjust based on user behaviour, data, and feedback.
Tools or techniques	Problem statement or brief Outcome hypothesis High-level value vs effort Stakeholder alignment	User interviews Simple prototypes Technical spikes or POCs Dependency mapping Assumption and risk lists	Initiative breakdown into thin slices Dependency and sequencing workshops, roadmap alignment Architecture or integration sketches Risk review sessions	Lightweight delivery plans Flow metrics Forecasting techniques (e.g. Monte Carlo) Scenario planning	Story mapping Acceptance criteria Three Amigos discussions (product, design, engineering) Technical and non-functional requirement notes	Feature flags/ releases Analytics and dashboards User feedback Incident, observability Retrospectives and learning
Anti Patterns	Agree feature list upfront Commit to delivery dates before any learning Assume the problem is fully understood	Skip discovery “to save time” Treat discovery as documentation or sign-off Push teams straight into build mode	Start work with unresolved dependencies Ask teams to “work around” known blockers Push risk decisions down to squads	Treat forecasts as fixed promises Ask for a detailed plan for the full initiative Optimise for certainty instead of learning	Write stories months in advance Over-specify or design everything upfront Treat tickets as contracts	Batch work into large releases Wait for approval instead of learning from real use Measure success only by delivery, not impact

Practices that drive predictability

Principle	Practices
Cross-Functional Leadership	Ideation, planning, and risk management.
Continuous Planning	Plan continuously based on learnings so that plans remain current
Lean Management	• Small batch sizes (incl. right sizing vs story pointing) • Manage WIP • Identify and reduce Waste
Engineering Excellence as an enabler	• Focus on quality • Healthy pipelines • Manage debt deliberately • Invest in tech
Outcomes Orientation	• Be clear on the why • Simplest way to achieve outcomes
Leverage data	• To inform the above

To enable ‘Elite’ performance within engineering squads, it's important to align on the behaviours that embody high performance and leverage metrics that promote those behaviours.

Metric	Definition (Industry standard)	Elite Target	Behaviour
Cycle time	**The elapsed time it takes for a ticket to be completed (In progress > Done)**	3 days	Small batch sizes/smaller stories, efficiency of ways of working
Throughput	**Number of tickets complete over a time box (weekly / sprint)**	Trend upwards	Optimise flow, identify and remove constraints
WIP	**Number of tickets in progress, started and not finished yet.**	X tickets per team	Optimise flow, promote collaboration, cognitive load mgmt
Deployment frequency	**How frequently are changes deployed**	Multiple per day	Faster Delivery
Lead time to change	**elapsed time from code commit to running in production**	< 1 hour	Healthy pipelines, faster delivery
Change Failure Rate	**The percentage of deployments that cause a failure requiring remediation.**	< 15%	Improve quality, shift left on quality

Metric	Planning	Daily	Review	Retrospective / Improve
Cycle Time	What is our expected cycle time or service level expectation (SLE) for this work?	Is any item at risk of breaching its SLE?	What was our actual cycle time? Did it meet expectations?	Where did time build up in the workflow? What slowed us down?
Throughput	What is a realistic amount of work we can complete given our capacity?	Are we completing work at a steady pace?	How much work did we complete? Was it in line with expectations?	What helped or hindered our ability to complete work?
Work in Progress (WIP)	What WIP limits are we committing to for this period?	Are we within WIP limits? If not, who is swarming to help?	Did WIP remain controlled throughout the period?	Where did WIP build up and why? How can we prevent this?
Work Item Age	Are any items already at risk due to size, uncertainty, or dependencies?	Is any item ageing unusually? Can we unblock, split, or refine it?	Which items took longer than expected to complete?	What signals did we miss earlier, and how can we respond sooner next time?
Flow Predictability	What level of confidence do we have in our delivery outlook?	Has anything changed that affects our confidence or forecast?	How accurate was our delivery outlook?	What would improve our confidence and predictability next time?
Blocked Time / Dependencies	What dependencies or risks do we expect going into this period?	Is anything blocked? Who owns resolving it?	How much time was lost to blockers or dependencies?	How can we surface and resolve dependencies earlier?

Data Driven Retrospectives

Set the Stage
An important step, often overlooked, to prompt focus, and create a safe space for honest inspection. This might be an icebreaker and/ or review of previous actions.

Gather Data
Build a shared view of what actually happened. Leverage data in this step to remind yourself of how the team fared in the last sprint or time box. Flow metrics or engineering metrics can be shared here.

Generate Insights
Explore patterns and strive to understand the causes behind the symptoms.

Decide What to Do
Select 1-3 of the biggest problems and pick actions or experiments that aim to resolve them.

Close the Session
Review what was agreed, and inspect the retrospective.

Questions and feedback….

- Does this align with your view of a target state?
- What else would you find valuable in a playbook?
- What challenges do we face in moving towards this target state?
- What is the best way to roll out initial changes?